

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«САМАРСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИМЕНИ АКАДЕМИКА С.П. КОРОЛЕВА»

Лабораторная работа №6
Многопоточное вычисление интегралов функций

Выполнила:

Иванова Елизавета Вадимовна,
студентка группы 6301-030301D

Самара 2025

Задание 1. Численное интегрирование методом трапеций

В классе Functions реализован статический метод:

```
public static double integral(Function function, double leftX, double rightX, double dx)
```

- Метод разбивает отрезок $[a, b]$ на подынтервалы длиной dx (последний — остаточный).
- На каждом подынтервале вычисляется площадь трапеции.
- Добавлена проверка: если a или b выходят за границы области определения функции — выбрасывается исключение.
- Для функции $f(x)=e^x$ на $[0, 1]$ установлено: при $dx=10^{-8}$ достигается точность до 7 знака после запятой.

Задание 2. Последовательная реализация (nonThread)

- Создан метод nonThread(), формирующий 100 заданий:
 - функция: логарифм с основанием $\in [1, 10]$,
 - $x_{\text{left}} \in [0, 100]$, $x_{\text{right}} \in [100, 200]$, $dx \in [0, 1]$.
- Все задания обрабатываются последовательно, без потоков.
- Вывод: 100 пар строк Source ... / Result

Задание 3. Простая многопоточная реализация (simpleThreads)

- Реализованы классы:
 - SimpleGenerator (генератор),
 - SimpleIntegrator (вычислитель).
- Оба потока синхронизируются на объекте Task через:
 - synchronized (task),
 - wait() / notify().
- Проблема: при первом запуске наблюдались:
 - NullPointerException (интегратор читал null),
 - "перемешивание" параметров (чтение половины данных от одного задания, половины — от другого).
- Решение:

Генератор ждёт, пока `function == null`, затем заполняет все поля.

Интегратор ждёт, пока `function != null`, читает всё, затем сбрасывает `function = null` и уведомляет.
- Результат: 100 корректных пар, без повторов и ошибок.

Задание 4. Реализация с собственным семафором (complicatedThreads)

- Создан класс SimpleSemaphore — бинарный семафор на основе wait()/notify().
- Изначально использовался один семафор, но это привело к гонке: оба потока конкурировали за захват, и данные перезаписывались.
- Решение: применена классическая схема Producer–Consumer с двумя семафорами:
 - empty — сигнал, что Task пуст и можно писать,
 - full — сигнал, что Task заполнен и можно читать.
- В main() семафор full изначально занят, чтобы интегратор ждал первую запись.

- Реализовано корректное прерывание:
 - после 50 мс вызывается interrupt() для обоих потоков,
 - в run() при InterruptedException поток немедленно завершается (return).
- Результат:

Строго поочерёдный вывод Source → Result,

все задания уникальны,

нет потери данных,

программа устойчива к изменению приоритетов

```
Source 71.58160136897006 136.02471421827113 0.693039315802954
Result 71.58160136897006 136.02471421827113 0.693039315802954 151.36871038876828
Source 55.870337792750306 192.01346611656686 0.2056438034516206
Result 55.870337792750306 192.01346611656686 0.2056438034516206 7903.55392461497
Source 24.656322937188403 157.13553549782335 0.019123051234744337
Result 24.656322937188403 157.13553549782335 0.019123051234744337 505.79285160527553
Source 68.39220644378975 166.0532305901487 0.6082425933324583
Result 68.39220644378975 166.0532305901487 0.6082425933324583 374.58431955561355
Source 36.47712120016142 147.9404052831307 0.43637563150150793
Result 36.47712120016142 147.9404052831307 0.43637563150150793 263.5073033688225
Source 47.3608134202699 172.75137987149864 0.7096972668220075
Result 47.3608134202699 172.75137987149864 0.7096972668220075 428.77197223612177
Source 79.77961189716835 113.8428016809792 0.9372194393840207
Result 79.77961189716835 113.8428016809792 0.9372194393840207 1645.2874775286523
Source 12.998397253943416 111.54621701747006 0.8274581849513131
Result 12.998397253943416 111.54621701747006 0.8274581849513131 208.32484855518405
Source 5.12408849671524 197.38680948388702 0.40506538406551584
Result 5.12408849671524 197.38680948388702 0.40506538406551584 709.6632728050258
Source 72.87264121131646 191.86636158872147 0.6145446371907333
Result 72.87264121131646 191.86636158872147 0.6145446371907333 2620.707870849713
Source 63.61289900326669 120.07622385260794 0.09321092668874142
Result 63.61289900326669 120.07622385260794 0.09321092668874142 182.68315401696418
Source 63.61289900326669 120.07622385260794 0.09321092668874142
Result 63.61289900326669 120.07622385260794 0.09321092668874142 182.68315401696418
Source 91.07706736522167 198.87866902384496 0.24184274637967873
Result 91.07706736522167 198.87866902384496 0.24184274637967873 243.11171426030768
Source 39.81901405152186 136.9927054693004 0.19901394352548019
Result 39.81901405152186 136.9927054693004 0.19901394352548019 414.18484003865944
Source 30.317928996967137 155.0430534402223 0.8338820567847316
Result 30.317928996967137 155.0430534402223 0.8338820567847316 370.8279231766503
Source 76.56639487949528 176.47866099176815 0.41224882280155306
Result 76.56639487949528 176.47866099176815 0.41224882280155306 737.0412962795074
Source 61.048207871412544 162.23980219660405 0.16199753846793452
Result 61.048207871412544 162.23980219660405 0.16199753846793452 223.3515406718879
Source 98.20278802558519 162.15041721346466 0.566550971428199
Result 98.20278802558519 162.15041721346466 0.566550971428199 143.34415677738053
Source 56.7893724736835 134.2350424505083 0.8487830594189214
Result 56.7893724736835 134.2350424505083 0.8487830594189214 177.2194597398969
Generator was interrupted
Integrator was interrupted

Process finished with exit code 0
```

Важное замечание:

Из-за прерывания через 50 миллисекунд программа не успевает обработать все 100 заданий — выводится только часть пар (например, 15–20).

Это соответствует условию задания, которое требует именно такого поведения.

Для проверки полной обработки всех 100 заданий достаточно убрать вызовы `interrupt()` и `Thread.sleep(50)`, заменив их на `join()` — в этом случае вывод строго содержит 100 уникальных пар `Source/Result` без повторов и перемешивания.

Выводы

- Метод трапедий успешно реализован и протестирован.
- Примитивы `wait()/notify()` позволяют организовать чередование потоков, но требуют строгого контроля состояния (`function = null`).
- Использование собственного семафора обеспечивает надёжную синхронизацию.
- Схема с двумя семафорами (`empty/full`) — наиболее надёжное решение для задачи "один производитель — один потребитель".
- Все режимы работы (`nonThread`, `simpleThreads`, `complicatedThreads`) протестированы и корректны.